

Supplementary Material for “A Mechanically Verified Translation of $\text{MTL}_{0,\infty}$ into Timed Büchi Automata”

Elie Fares, Jean-Paul Bodeveix, Hussain Al-Aqrabi, and Azar Salami



This supplement contains the detailed reset-policy and residual-obligation arguments, reset-completion proofs, and auxiliary clock arithmetic. The Rocq source files used by the development are included in the accompanying `proof/` directory. The complete project and build instructions are at <https://github.com/efares1/coq-verified-mtl-to-tba>. Cross-references point to the main paper.

For completeness, Theorem 5.1 supplies a clock-consistent extension of w satisfying $\mathcal{T}(f)$. The backend contract therefore gives an accepting run of the original backend automaton. Relaxation preserves this run, and the reset-completion lemma supplies an extension accepted by the completed TBA. The mechanized theorem is `MTL_to_TBA_correct_with`.

1 OMITTED PROOFS FOR THE TBA CONSTRUCTION

1.1 Reset Completion

Proof of Lemma 4.2. The extension ρ' is clock-consistent by construction. Consider a preservation clock x . If the cube taken at position i contains `unch( $x$ )`, then neither ρ nor ρ' resets x at i ; otherwise ρ' resets it. Hence ρ' resets x wherever ρ does, and the clock recurrence (9) gives, by induction on positions, $\text{val}_{\rho'}(i, x) \leq \text{val}_{\rho}(i, x)$. The literals over x are upper bounds, so remain true. Dually, ρ' resets a restart clock only where the cube contains `rst( $x$ )`, hence only where ρ resets it. Therefore $\text{val}_{\rho'}(i, x) \geq \text{val}_{\rho}(i, x)$, and the lower-bound literals remain true. Action literals and the Büchi condition are unchanged, so the same run is accepting after completion (Rocq lemma `complete_complete`).

1.2 Exactness of Relaxation and Reset Completion

Proof of Proposition 4.3. For soundness, a run of the completed TBA gives a run of the relaxed automaton on the propositional word of its clock-consistent extension. At each position, the relaxed transition comes from an original backend cube. Reconstruct a valuation satisfying that cube by setting each relevant extended atom false when its negative literal was deleted, retaining values required by surviving literals, and choosing values for deleted irrelevant atoms as needed. The resulting word follows the same state sequence in the original backend automaton, so the backend contract makes it a model of $\mathcal{T}(f)$. The reconstructed and relaxed words agree on base atoms; on relevant extended atoms, relaxation can only change false to true. Since $\mathcal{T}(f)$ contains extended atoms only positively, it is upward closed in those atoms, so the relaxed word also satisfies $\mathcal{T}(f)$. The soundness direction of Theorem 5.1 gives $w, 0 \models_{\text{MTL}} f$.

2 DETAILED PROOFS OF SOUNDNESS AND COMPLETENESS

The main paper states the semantic model, translation clauses, residual predicates, canonical reset policy, and proof roadmap. This supplement gives the omitted primitive soundness and completeness arguments, the structural induction, and the detailed theorem composition. It uses the main-paper definitions rather than restating them.

2.1 Soundness of the Primitive Timed Translations

For each primitive hatted timed constructor F , the local soundness lemma has the form

$$\rho, i \models_{\text{LTL}} \mathcal{T}_{\pi}(F) \implies \text{ew_base}(\rho), i \models_{\text{MTL}} F, \quad (1)$$

assuming clock consistency and the induction hypotheses for the immediate subformulas. No canonical reset-policy assumption appears in (1).

Upper-bounded hatted Until. The translated LTL Until starts at $i + 1$ and provides an endpoint $j > i$. At position i itself the translation does not constrain the clock, but the recurrence (9) together with nonnegativity gives $\text{val}_{\rho}(i + 1, x) \geq \Delta_i$ whether or not x is reset at i . Along the intermediate positions (i, j) the shared clock is unchanged, so Lemma 5.4 applied on $[i + 1, j)$ yields $\text{val}_{\rho}(j, x) \geq \Delta_i + \text{elap}_w(i + 1, j) = \text{elap}_w(i, j)$, and the endpoint guard $x \leq d$ gives

$$\text{elap}_w(i, j) \leq d.$$

The induction hypotheses turn the translated prefix $\bar{P}$ into p and the endpoint $\bar{Q}$ into q , matching the semantics of $p \hat{U}_{\leq d} q$. For $\hat{U}_{< d}$, the strict guard $x < d$ gives $\text{elap}_w(i, j) < d$ in the same way (`sound_UhatLt`).

Lower-bounded hatted Until. The translation resets x at i and follows Γ . In the finite branch, a future state satisfies

$x \geq d$ together with $\bar{P} \cup \bar{Q}$. By (13), the endpoint is at least d time units after i , and the nested Until supplies the required p -prefix and q -endpoint. In the second branch, $G\bar{P} \wedge GF\bar{Q}$ holds. Then p holds forever and q recurs infinitely often; time divergence guarantees a q occurrence at physical distance at least d . For $\bar{U}_{>d}$, the strict threshold $x > d$ gives a strict lower bound in the finite branch, and the fairness branch uses the strict argument of Section 5.2 (`sound_UhatGt`).

Upper-bounded hatted Release. The translation resets the shared clock and requires

$$\bar{Q} \text{ W } ((x > d) \vee (\bar{P} \wedge \bar{Q})).$$

Before the weak-until endpoint, q holds at every strictly future position. If the endpoint is $\bar{P} \wedge \bar{Q}$, p discharges the Release obligation; if it is $x > d$, the upper-bounded window has ended. For $\hat{R}_{<d}$, the window ends at $x \geq d$ (`sound_RhatLt`).

Lower-bounded hatted Release. The translation requires

$$((x < d) \wedge \text{unch}(x)) \text{ W } ((\bar{P} \text{ R } \bar{Q}) \vee ((x < d) \wedge \bar{P}))$$

from $i + 1$. While the left side holds, the clock is preserved and remains below d , so those positions are still before the lower threshold. At the endpoint either an ordinary untimed Release takes over or p discharges the obligation before the threshold is reached. For $\hat{R}_{>d}$, the pre-threshold test is $x \leq d$ (`sound_RhatGt`).

There are no separate ordinary timed soundness lemmas. After unfolding Eqs. (1)–(4), their correctness follows from the Boolean cases of the structural induction and the eight hatted lemmas above.

2.2 Completeness under the Canonical Reset Policy

The following case arguments prove the converse on the canonical extension defined in the main paper:

$$w, i \models_{\text{MTL}} F \implies \rho^*, i \models_{\text{LTL}} \mathcal{T}_\pi(F).$$

They use the reset decisions, clock update, and residual predicates already given in Table 3 and Eq. 9; the details below establish how each primitive case satisfies its translated obligation.

Upper-bounded hatted Until. Choose the first q -position among the MTL witnesses. The proof maintains, for every position n with $i < n \leq j$ up to that endpoint j , the invariant

$$\text{val}_{\rho^*}(n, x) + \text{elap}_w(n, j) \leq d.$$

It need not hold at the activation position i itself (the policy may reset there because the old value is useless); it is established at $i + 1$ in both the reset and the preservation case. If the policy preserves the clock, Table 3 guarantees an older reference whose residual URes is still satisfiable strictly after the current position; if it resets, the new value is consistent with the current activation. Hence every intermediate position satisfies the clock guard required by the translated Until and the first q -position supplies its endpoint. This is the content of the mechanized lemma `complete_UhatLe`. The strict case maintains the strict invariant $\text{val}_{\rho^*}(n, x) + \text{elap}_w(n, j) < d$ with the residual $\text{URes}^<$ (`complete_UhatLt`).

Lower-bounded hatted Until. At every position where F holds, the canonical policy resets the shared clock (Table 3). The proof propagates an active obligation until a position satisfies

$$x \geq d \quad \text{and} \quad \bar{P} \cup \bar{Q},$$

which yields the finite branch of Γ . If no such position is reached, the active-obligation invariant implies that p remains continuously true, and since every later position again carries a pending obligation that requires a later q , the proposition q recurs infinitely often. This gives

$$G\bar{P} \wedge GF\bar{Q},$$

the second branch of Γ . This is formalized by `complete_UhatGe`, and by `complete_UhatGt` for the strict threshold $x > d$.

Upper-bounded hatted Release. The canonical policy resets the clock at every position where F holds. The proof follows the currently active nontrivial Release obligation. Until it is discharged, the clock measures elapsed time from the relevant start and q continues to hold. The weak until terminates either because $p \wedge q$ discharges the Release or because $x > d$, meaning that the upper-bounded window has ended. This is the role of `complete_RhatLe`, and of `complete_RhatLt` for the strict window end $x \geq d$.

Lower-bounded hatted Release. Here the invariant is

$$\text{RRes}(n, \text{val}_{\rho^*}(n, x), d, p, q).$$

By Table 3, the shared clock is preserved exactly while $x < d$, the residual holds, and p does not occur; in that case the residual shifts from (i, v) to

$$(i + 1, v + \Delta_i).$$

The Rocq lemma `rge_residual_shift` establishes this one-step update. A converse lemma on the preservation policy recovers the residual information needed at the next position, so the left side of the weak until remains valid until either an ordinary untimed Release is established or p occurs while the clock is still below d . This yields `complete_RhatGe`. The strict case uses $\text{RRes}^>$, the pre-threshold test $x \leq d$, and the shift lemma `rgt_residual_shift` (`complete_RhatGt`).

As with soundness, ordinary timed operators need no additional completeness lemmas: unfolding their definitions reduces them to Boolean structure and one of the eight hatted primitive cases above.

2.3 Structural Equivalence at Every Syntactic Occurrence

The local timed lemmas are assembled with the homomorphic Boolean and untimed temporal cases into the following stronger induction theorem.

Theorem 2.1 (Canonical correctness at every occurrence). *Let root be well formed and suppose*

$$\text{node}(\text{root}, \pi) = f.$$

Then for every position i ,

$$w, i \models_{\text{MTL}} f \iff \rho^*, i \models_{\text{LTL}} \mathcal{T}_\pi(f),$$

where $\rho^* = \text{canonical_ext}(\text{root}, w)$.

Proof idea. The proof is by structural induction on the formula f . Atomic propositions, Boolean connectives, Next, untimed Until, and untimed Release follow directly from their semantics and the induction hypotheses. Since the ordinary timed operators are Rocq definitions, they have already unfolded into these structural cases. The only primitive timed branches are therefore the eight hatted constructors. In each such branch, the forward direction invokes the corresponding completeness lemma from Section 2.2, while the reverse direction invokes the corresponding soundness lemma from Section 2.1. The node lemmas locate the two children, and well-formedness of the located formula provides the bound and recursive hypotheses. $\square$

This is stronger than a root-only statement and is important in the mechanized proof. The induction retains the path in order to locate the current subformula and its children, but the path is not the clock key. If two paths locate the same primitive timed formula, both branches refer to the same formula-keyed clock.

2.4 Proof of the Correctness Theorem

We now prove Theorem 5.1.

Proof. We prove the two implications separately.

$(\Rightarrow)$ *MTL to clocked LTL.* Assume $w, 0 \models_{\text{MTL}} f$. Choose the canonical extension

$$\rho = \text{canonical_ext}(f, w).$$

By construction, ρ has the same base timed word as w and is clock-consistent. Applying Theorem 2.1 at the root path $[]$ and position 0 gives

$$\rho, 0 \models_{\text{LTL}} \mathcal{T}(f).$$

Therefore the required extension exists.

$(\Leftarrow)$ *Clocked LTL to MTL.* Assume there exists an arbitrary ρ such that

$$\begin{aligned} &\text{same_base}(\rho, w), \quad \text{clock_consistent}(\rho), \\ &\rho, 0 \models_{\text{LTL}} \mathcal{T}(f). \end{aligned}$$

This extension need not be canonical, so completeness cannot be used. Instead, a structural induction on f uses the policy-independent soundness lemmas of Section 2.1 and obtains

$$\text{ew_base}(\rho), 0 \models_{\text{MTL}} f.$$

Finally, $\text{same_base}(\rho, w)$ rewrites the underlying timed word $\text{ew_base}(\rho)$ to w , yielding $w, 0 \models_{\text{MTL}} f$. $\square$

3 ROCQ RESULT MAP

The paper's principal statements correspond to these definitions and theorems in the included Rocq development.

Proof source. The Rocq definitions of `EncodingCorrect_proved` and `MTL_to_TBA_correct_with` are in `proof/EncodingCorrect_Shared_Clock_Derived_Strict_Direct_Proof.v`.

Core source. The definitions of `T_uses_every_clock` and `complete_complete` are in `proof/MTL_to_TBA_Shared_Clock_Derived_Strict_Direct_Core.v`.

The core file also contains the four ordinary Until and Release compatibility lemmas and their strict variants.

The theorem with suffix `_with` assumes the back-end correctness contract as an argument. The development includes no project-specific assumption in that theorem beyond this explicit contract. The standard library assumptions are preserved in `ASSUMPTIONS.txt`; reproduction instructions are in `README_proof.md`. The convenience theorem `MTL_to_TBA_correct` uses the core axiom `LTL_TO_BUCHI_CORRECT`; the main paper states the contract-parameterized theorem.